

NLP Customer Reviews: notebook guide

Ironhack AI Engineering, project 4 · Hamza Shabbir & Yasaman Najafi Jozani · what every notebook and cell does, in short

The big picture

Each notebook saves a file and the next one reads it, like stations in a kitchen: **01** washes the vegetables (cleaning), **02** learns to taste good from bad (sentiment), **03** sorts dishes onto shelves (categories), **04** writes the menu (articles), and the **app** serves customers.

Har notebook ek file save karta hai aur agla notebook usi ko parhta hai, jaise kitchen ke stations.

01 clean -> 02 sentiment -> 03 clusters -> 04 articles -> app demo

Folder	What lives there
src/	Shared helpers: config.py (all file paths), preprocessing.py (load + clean functions)
data/raw, data/processed	Original Kaggle CSV, then the cleaned and clustered CSVs (not on GitHub)
notebooks/	01 to 04, run in order
app/	app.py (Gradio), model_config.json (which model), text_cleaning.py, requirements.txt
outputs/	figures, articles (Task 3) and metrics (model comparison)

Notebook 01 · EDA and preprocessing

Look at the raw data, find the problems, and save one clean file.

Think of it as: unpacking a grocery delivery: count items, throw away damaged ones, remove packaging.

Kacha data dekho, masle dhoondo, saaf file save karo.

Cell	What it does	Real-life example
1	Imports pandas and the helpers from src/.	Opening your toolbox.
2	load_raw() reads the CSV: 34,660 rows x 21 columns.	Unloading the delivery boxes.
3	isna().sum() counts empty values per column.	6,760 reviews have no product name.
4	Drops 15 columns we don't need (ids, user city, dates...).	Throwing away the packaging.
5	nunique() counts distinct values.	48 product names, but some are two glued together.
6	Checks missing values again after the drop.	A second look before cooking.
7	preprocessing() drops rows missing rating, text, name or title, removes duplicate and empty reviews, saves the file.	34,660 reviews become 27,863 clean ones.

Reads: data/raw/1429_1.csv **Writes:** data/processed/cleaned_1429_1.csv

Notebook 02 · Task 1: sentiment analysis

Teach a model to read a review and say negative, neutral or positive. Compare models fairly and save the winner for the app.

Think of it as: a student studies past exams (train set) and is graded on a new exam (test set).

Model purane reviews se seekhta hai aur naye reviews pe test hota hai.

Cell	What it does	Real-life example
1	Imports and loads the clean CSV.	-
2	Stars to labels (1-2 bad, 3 neutral, 4-5 good). Splits 70% train / 30% test with stratify.	Both exams get the same mix of easy and hard questions.
3	Removes HTML tags.	 Great tablet becomes Great tablet.
4	Lowercase, letters only, no single letters.	"Great!! 10/10" becomes "great".
5	Removes stopwords (the, is, and...).	Also removed "not": the bug we found later.
6	Lemmatizes words to their base form.	"tablets" becomes "tablet".
7	TF-IDF: each review becomes 5,000 word scores. Fit on train only. Shows top words per class.	A highlighter: words common in this review but rare overall glow brightest ("slow" in bad reviews).
8	Better TF-IDF: word pairs, ignore rare words, softer counts.	"not good" is kept as one clue.
9-11	Naive Bayes (macro-F1 0.46), Logistic Regression (0.54), LinearSVC (0.52), with class weights.	Three students take the same exam.
12	Text: why the first app failed.	"Worst customer service" was called positive.
13	Title + text as input, same split, 9 hand-written test sentences, an evaluate() helper.	Adding the headline gives extra clues.
15	TF-IDF v2 Pipeline: clean (keeps not/no/too) -> TF-IDF -> LogReg C=10. Macro-F1 0.61.	One machine: wash, chop, cook in one go.
17	RoBERTa zero-shot: pretrained, no training, predicts the test set.	Hiring an experienced chef instead of training a new one.
19	Comparison table + hand tests: TF-IDF 5/9, RoBERTa 9/9.	Exam score plus a real-world interview.
21	CHOICE = "transformer"; saves the backup model, writes model_config.json and requirements.txt.	Writing the hiring decision on the kitchen door.

Reads: cleaned_1429_1.csv **Writes:** app/model_config.json, app/requirements.txt, models/, outputs/metrics/

Why macro-F1? 93% of reviews are positive, so a model that always says "positive" gets 93% accuracy and is useless. Macro-F1 averages the score of each class, so negative and neutral count as much as positive.

93% reviews positive hain, isliye accuracy dhoka deti hai. Macro-F1 har class ko barabar ahmiyat deta hai.

Notebook 03 · Task 2: product clustering

Group 40 products into 5 categories without labels, then name each group.

Think of it as: sorting a pile of laundry: nobody tells you the piles, you group by what the items are.

Bina labels ke products ko groups mein baanto, phir har group ka naam rakho.

Cell	What it does	Real-life example
1	Imports pandas, KMeans, PCA, TF-IDF and src/.	-
3	Loads the clean reviews (27,863).	-
5	<code>clean_product_name()</code> keeps the first of two glued names.	"Echo (Black),,Charger" becomes "Echo (Black)".
7	<code>groupby + agg</code> : one row per product with merged category tags, review count, average stars, and a description sentence.	3 Echo reviews become 1 row: Echo (White), 3 reviews, 4.33 stars.
9	Sentence-transformer turns each description into 384 numbers that capture meaning.	GPS coordinates for meaning: "Fire tablet" and "Kindle Fire" land close together.
11	Tries $k = 2$ to 8; shows silhouette (separation) and group sizes.	Trying 4, 5 or 6 laundry piles to see which looks tidiest.
13	KMeans with $K = 5$ gives each product a cluster number.	Every item goes to its nearest pile.
15	Per cluster: top TF-IDF words and every product.	Cluster 0: kindle, paperwhite, ereader.
17	PCA squeezes 384 numbers into 2 for a scatter plot.	Turning a globe into a flat map.
19	You replace "Cluster 0" with a real name.	"Kindle e-readers".
21	Optional check against your own expected groups (Adjusted Rand Index).	A teacher checks your sorting.
23	Adds the category to every review and saves.	Labelling the shelves.

Reads: `cleaned_1429_1.csv` **Writes:** `reviews_clustered.csv`, `product_clusters.csv`, `outputs/figures/`

Notebook 04 · Task 3: AI buyer's guides

Write one short article per category. Pandas finds the facts, a small LLM writes them up.

Think of it as: a journalist collects notes first, then writes the article from the notes, not from memory.

Pehle facts pandas se, phir LLM sirf likhta hai. Is tarah LLM jhoot nahi banata.

Cell	What it does	Real-life example
1	Imports; picks file paths for Colab or your Mac.	Same recipe, different kitchen.
2-3	Colab path tests (current folder, Drive write test). Can be deleted.	-
5	Loads reviews_clustered.csv; warns if categories still say "Cluster N".	-
7	Per product: reviews, average stars, % negative. Products under 30 reviews are not ranked.	Don't trust a restaurant with 5.0 stars from 3 reviews.
9	Facts sheet per category: top 3, weakest product, real praise and complaint quotes.	The journalist's notebook.
11	Loads Qwen2.5-1.5B-Instruct; generate() sends a chat and returns the answer (no randomness).	Hiring a junior writer.
13	3 prompt versions: basic, structured, structured + example article.	Three briefs given to the writer.
15	Runs all 3 prompts on the biggest category.	Test draft before the full job.
17	Scores drafts: length, sections found, top 3 named, invented ratings.	A fact-checker reads every draft.
19	Best prompt (structured) writes all 5 articles and saves them with their facts sheet.	Publishing the guides.
20-22	Final check table; Colab zip download; note on optional fine-tuning.	-

Reads: reviews_clustered.csv **Writes:** outputs/articles/*.md

The app · Task 4: deployment

A Gradio web page: paste a review, get three confidence scores.

Think of it as: the restaurant counter. The kitchen work is done; this just serves.

App sirf serve karti hai; asal kaam notebooks mein ho chuka.

File	What it does	Real-life example
model_config.json	Says which model to serve: transformer or tfidf.	The menu board says today's chef.
app.py	Reads the config, loads that model, shows the text box and scores.	"Worst customer service" -> negative 0.9.
text_cleaning.py	Same cleaning as training, used by the TF-IDF backup model.	Same recipe at home and in the restaurant.
requirements.txt	Libraries for Hugging Face Spaces: gradio, transformers, torch.	The shopping list.

Run: cd app then python app.py -> http://127.0.0.1:7860